

Logbuch — KI1-Projekt 308

Name: Felix Hollfoth **Gruppe:** 308 **Matrikelnummer:** 8221324

Eintrag 1

Datum: 16.02.2026 **Titel:** Repository einrichten und Ideen für Gruppeneinteilung aufstellen **Aus Gruppentreffen:** Gruppentreffen 1 (11.02.2026)

Arbeitsschritte:

- VS Code gedownloadet und Repository geklont
 - GitHub eingerichtet und Features installiert (pdf viewer, KI etc.) [11]
 - Python 3.14.3 installiert
 - nbstripout heruntergeladen und aktiviert, damit Ausführungen von Notebooks nicht committed werden
 - In den California Housing Datensatz eingearbeitet und sich das Notebook aus der Vorlesung angeschaut
 - Vorlesungsfolien nach relevanten Methoden untersucht
 - Übergordnete Aufgaben zusammengetragen (Datenbereinigung, -analyse, -transformation, Lernen des neuronalen Netztes (layers, units, Aktivierungsfkt. etc., Prüfen auf Overfitting, Validierungsmethoden, Parameterreduzierung, Regularisierung, Vergleich mit linearer Regression) für Vorstellung in nächstem Gruppentreff um die Organisation zu erleichtern
-

Eintrag 2

Datum: 22.02.2026 **Titel:** Programmierumgebung geschaffen und Datenskalisierung **Aus Gruppentreffen:** Gruppentreffen 1 (11.02.2026) und Gruppentreffen 2 (17.02.2026)

Arbeitsschritte:

- Auf Hinweis von Gruppenmitglied Kolja musste Python 3.14 für tensorflow gedowngradet werden. Nach Recherche geht dies nur mit Python 3.10 oder

niedriger, daher Python 3.10.13 über pyenv installiert und bei VS Code als Interpreter und Kernel festlegen

- Funktionen geschrieben um den Datensatz zu Skalieren mittels MinMaxScaler und StandardScaler (aus Gruppentreff 2 17.02.2026)
 - wichtig da einige Features sehr große und andere sehr kleine Werte haben, kann es sonst zu Problemen beim Gradientenverfahren (Backpropagation) geben
 - Skalierung kann auf die Aktivierungsfunktion abgestimmt werden
 - Funktionen resistent gemacht gegen die Eingabe von DataFrames und Array -> Eingabedatentyp wird auch wieder ausgegeben
 - target_values werden nicht skaliert für die Stabilität und Interpretierbarkeit des Netzes
-

Eintrag 3

Datum: 02.03.2026 und 03.03.2026 **Titel:** virtuelle Pythonumgebung neu aufgesetzt

Aus Gruppentreffen: Gruppentreffen 4 (27.02.2026)

Arbeitsschritte:

- mit der installierten Tensorflow Version aus Eintrag 2 konnte 05_flo_NN.ipynb nicht ausgeführt werden. Numpy $\geq 1.21.1$ lies sich unter python < 3.11 nicht installieren.
 - Bei dem wechsel auf Python 3.11. wurde .git verändert und das Repro musste neu geclost werden
 - Durch Konflikte mehrerer Pythonversionen, .venv und .pyenv Konfigurationen, wurde alles komplett gelöscht und ein neues Setup erstellt:
 - TensorFlow Version: 2.10.1
 - Numpy Version: 1.26.4
 - Pandas Version: 2.3.3
 - Matplotlib Version: 3.10.8
 - Python Version: 3.10.11
-

Eintrag 4

Datum: 07.03.2026 **Titel:** Probleme mit Not a Number (NaN) beim erstellen des Neuronalen Netzes **Aus Gruppentreffen:** Gruppentreffen 4 (27.02.2026)

Arbeitsschritte:

- Beginn Programmierung (mit den Aufgaben aus Gruppentreffen 4)
 - Hilfsfunktionen werden von Florian genutzt [3]
 - Daten werden damit geladen, bereinigt und skaliert, sodass für die folgende Analyse Standardisierte-, Normalisierte Daten von null bis eins und die originalen Daten zur Verfügung stehen
 - Beim erstellen eines ersten Neuronalen Netzes kam der Fehler NaN (Not a Number). Fehlerursache (Vermutungen):
 - Das Dataframe ist nach dem skalieren intern beschädigt, trotz bzw. wegen eingebauter Rückumwandlung
 - Beim konvertieren in Numpy entstehen echte NaNs, die im Dataframe enthalten bleiben
 - Gruppenmitglied Florian hat in der Funktion: `get_train_test_split()` [3] ebenfalls einen scaler miteinprogrammiert, welcher diesen Fehler nicht enthält, weshalb dieser ab sofort genutzt wird
-

Eintrag 5

Datum: 07.03.2026 und 08.03.2026 **Titel:** Bestimmung von Hyperparametern (Aktivierungsfunktion und Skalierung) **Aus Gruppentreffen:** Gruppentreffen 4 (27.02.2026)

Arbeitsschritte:

- Um die Arbeitsergebnisse vergleichen zu können wird ein Netz als Basis trainiert mit Parametern aus der Literatur [1] und von Florian [3]
- Da die Konvergenz eines Verfahrens häufig sehr stark von dem Wertebereich und der dazugehörigen Aktivierungsfunktion abhängt wird über folgende Kombinationen iteriert:

- Aktivierungsfunktionen: 'relu', 'tanh', 'sigmoid', 'elu', 'selu', 'leaky_relu'
- Datenskalisierung: keine, standard, min0_max1

Ergebnisse:

R^2-Score Train					
	Aktivierungsfunktion	normal	standard	minmax	Durchschnitt
	relu	-0,1400	0,8171	0,7856	0,4876
	tanh	0,306	0,8045	0,7493	0,6199
	sigmoid	0,5693	0,7643	0,6852	0,6729
	elu	0,5623	0,7674	0,7245	0,6847
	selu	0,2391	0,7781	0,7546	0,5906
	leaky_relu	-0,5903	0,7905	0,7671	0,3224
R^2-Score Test					
	Aktivierungsfunktion	normal	standard	minmax	Durchschnitt
	relu	-0,1504	0,7561	0,7618	0,4558
	tanh	0,309	0,7668	0,7405	0,6054
	sigmoid	0,5689	0,7529	0,6786	0,6668
	elu	0,559	0,7479	0,7165	0,6745
	selu	0,2315	0,7577	0,7462	0,5785
	leaky_relu	-0,5892	0,7564	0,7541	0,3071
Differenz zwischen Train und Test Score:					
	Aktivierungsfunktion	normal	standard	minmax	
	relu	0,0104	0,061	0,0238	
	tanh	-0,003	0,0377	0,0088	
	sigmoid	0,0004	0,0114	0,0066	
	elu	0,0033	0,0195	0,008	
	selu	0,0076	0,0204	0,0084	
	leaky_relu	-0,0011	0,0341	0,013	

- Außer den in den Vorlesung bekannten Aktivierungsfunktionen wird selu mitberücksichtigt, da sie beim Durchlauf die Aktivierungen normalisieren kann. Vorallem für tiefere Netze interessant, falls mehr als zwei hidden layer sich als nützlich erweisen sollten [2]
- Standardisierung laut Tabelle die beste Option. Dahingehen sorgt keine Skalierung dafür, dass einige Modelle je nach Aktivierungsfunktion sogar schlechter sind als der Mittelwert. Das kann an den großen Unterschieden im Wertebereich der Features liegen
- Bester Test-Score zeigt sich beim tanh mit der Standardisierung mit 0,7668. Für künftige Modelle verwenden.

- Abweichung von Test- zu Trainingsscore liegt bei 3,8 %, welches leichtes overfitting andeutet aber als noch nicht problematisch angesehen wird, daher erstmal keine weitere Berücksichtigung

Eintrag 6

Datum: 09.03.2026 **Titel:** Bestimmung von Hyperparametern (Lernrate, Batch-Size) **Aus**

Gruppentreffen: Gruppentreffen 4 (27.02.2026)

Arbeitsschritte:

- Lernrate grob bestimmt (Test von $\alpha = 1E-6$ bis 10); Resultat: zwischen 0,001 und 0,01 die einzigen Bereiche, in denen der Score auf den Testdaten über 70 % ist

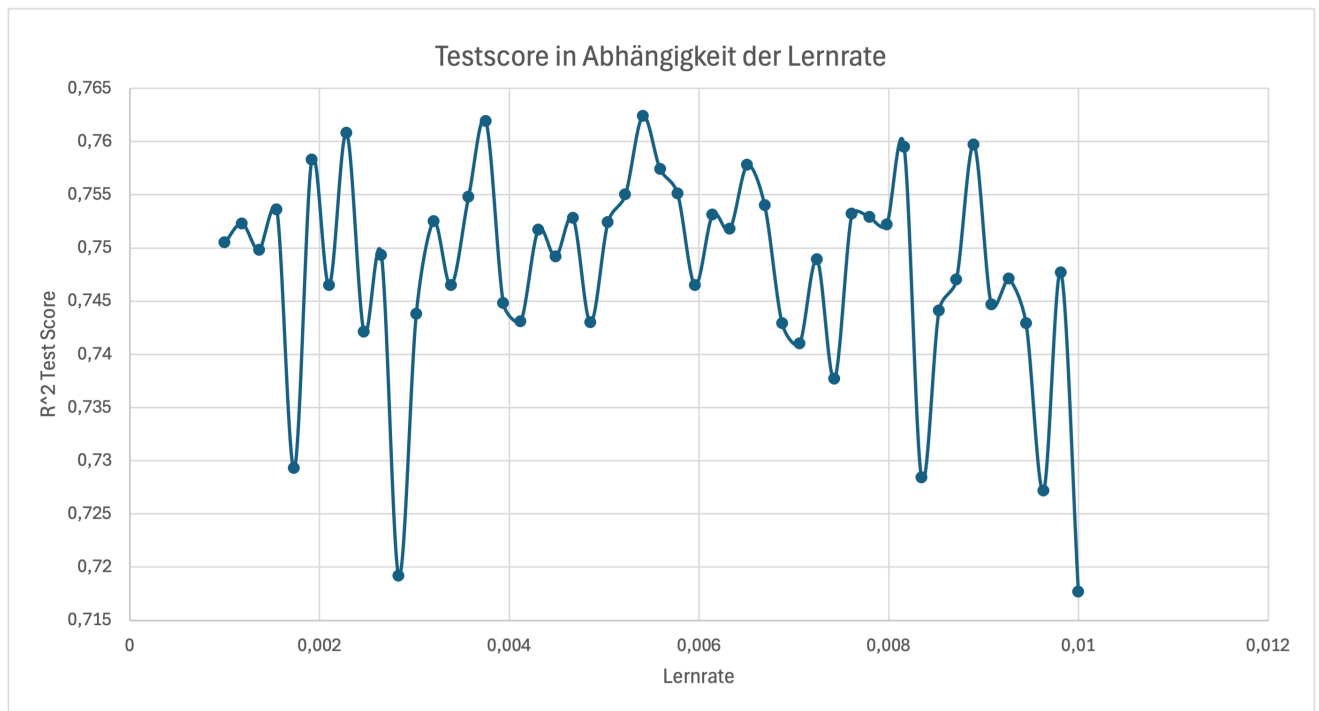
```
=====
NN_LearningRate_0.001
=====
R² Score:  Train = 0.8020  |  Test = 0.7672
MAE:       Train = 0.2977  |  Test = 0.3161
RMSE:      Train = 0.4270  |  Test = 0.4599
=====

=====
NN_LearningRate_0.01
=====
R² Score:  Train = 0.8215  |  Test = 0.7455
MAE:       Train = 0.2900  |  Test = 0.3359
RMSE:      Train = 0.4054  |  Test = 0.4809
=====

=====
NN_LearningRate_0.1
=====
R² Score:  Train = 0.5307  |  Test = 0.5257
MAE:       Train = 0.5269  |  Test = 0.5259
RMSE:      Train = 0.6574  |  Test = 0.6565
=====

=====
NN_LearningRate_0.5
=====
R² Score:  Train = -0.5483  |  Test = -0.5856
=====
```

- Zwischen 0,001 und 0,01 in 50 identischen Schritten durchitteriert -> Ergebnisse schwanken sehr



- Lernrate konstant über 0,74 im Intervall [0,0032; 0,0072] und bester Einzelwert bei 0,0054 welcher bei dieser Schwankung aber auch deutlich auf Zufall im Modell hindeutet
- Schwankung bei erneuter fünfmaliger Ausführung mit identischen Hyperparametern von über 3 % erkannt -> Training der Hyperparameter liegt in diesem Schwankungsbereich -> Statistische Untersuchung der Schwankung bei erneuter Ausführung erforderlich, da sonst eine Verbesserung der Hyperparameter durch die große Varianz des Trainings keine Bedeutung zukommt

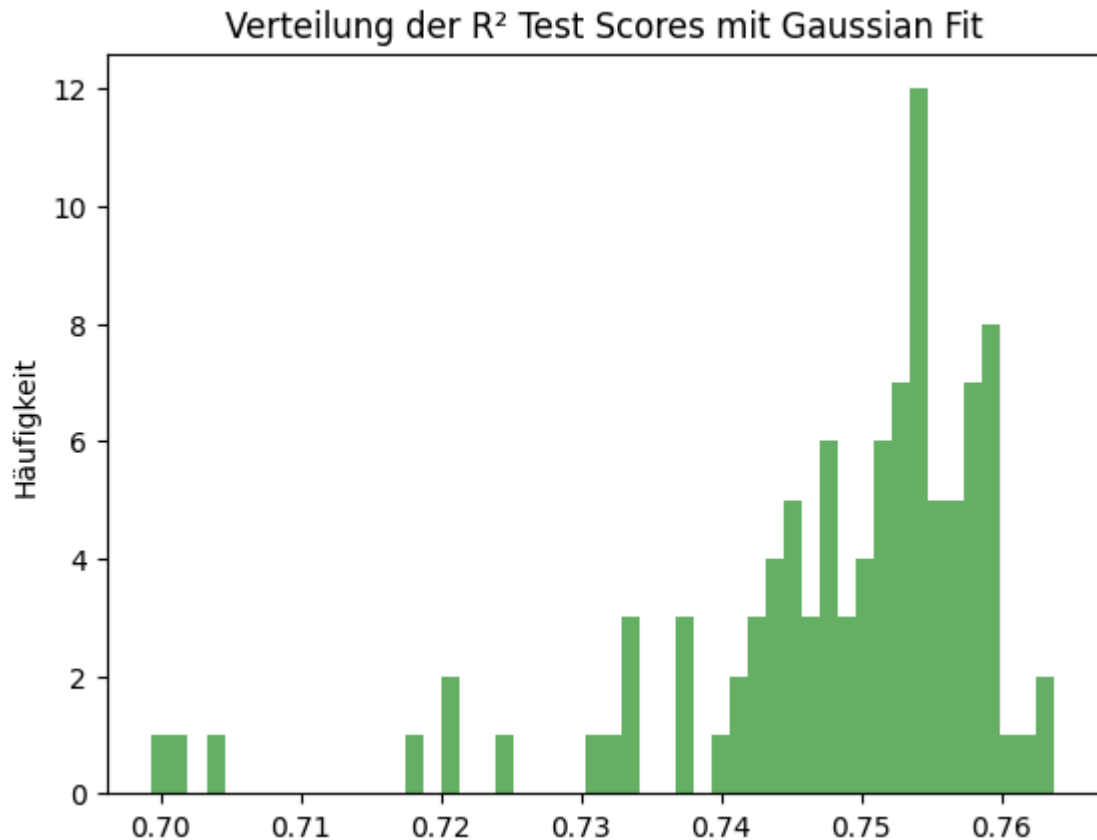
Eintrag 7

Datum: 10.03.2026 **Titel:** Bestimmung von Hyperparametern (Erneute Ausführungen, Epochenanzahl) **Aus Gruppentreffen:** Gruppentreffen 4 (27.02.2026)

Arbeitsschritte:

- Histogramm über 100 Modelle mit konstanten Parametern erstellt -> wenige Modelle 0,7 und 0,72; meisten Modelle liegen zwischen 0,74 und 0,765 [4]
 - Mittelwert: 0,7481
 - Varianz: 0,0002

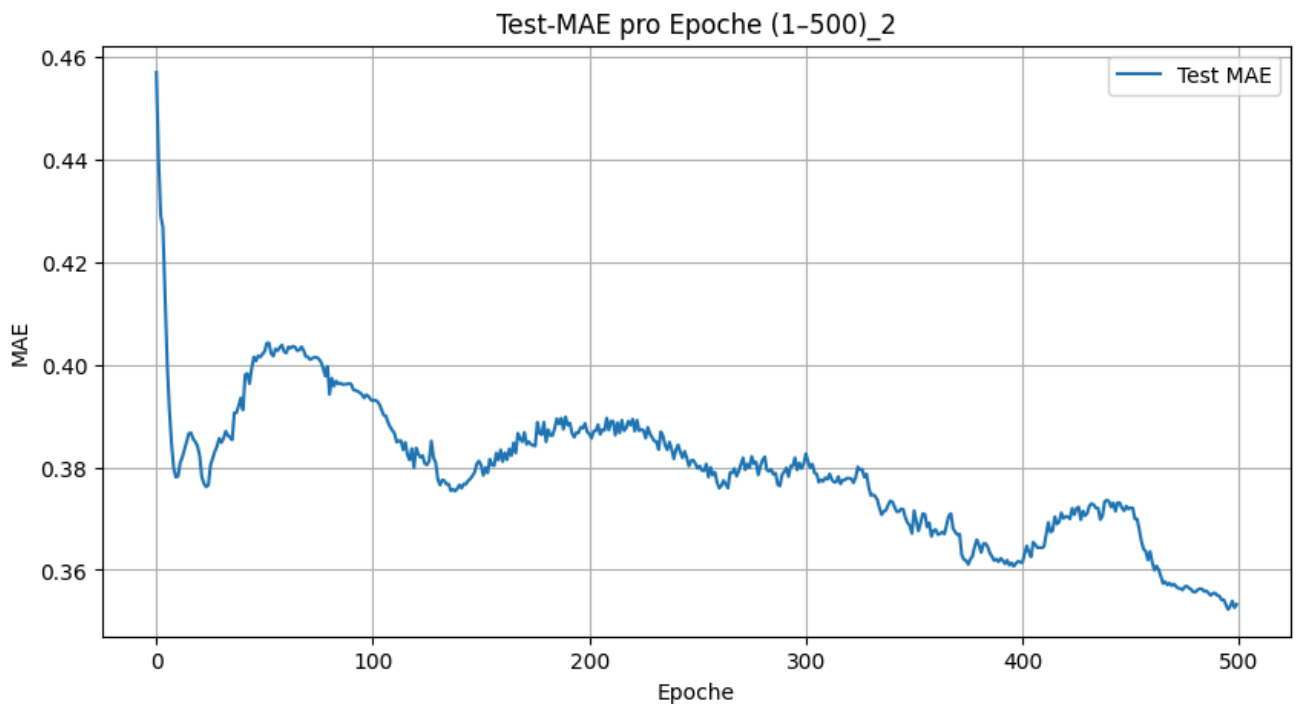
- Standardabweichung: 0,0124



- Standardabweichung zeigt, dass die bisher betrachtete Entwicklung von Hyperparameter genauso schwankt, wie der Testscore bei erneuter Ausführung -> neue Hyperparameter betrachten und deren Einfluss untersucht
- folgende batch-sizes werden Betrachten für eine grobe Einordnung: batch_size = [4, 8, 16, 32, 64, 128, 256, 512] -> zwischen einer batch-size von 16 bis 128 werden die Modelle alle stabil innerhalb der Standardabweichung trainiert
- Epochen betrachtet: epochs = [10, 50, 100, 200, 500, 1000, 10000] -> ab 500 Epochen wird Overfitting stärker; bis 500 Epochen verbessert sich der Test-Score -> Entscheidung einen Verlauf des MAE-Wertes auf den Testdaten pro Epoche ausgeben zu lassen Overfitting durch zu viele Epochen:

Epochenanzahl	Train Score	Test Score	MEA Test	RMSE Test
10	0,7002	0,6954	0,3841	0,5262
50	0,7381	0,7286	0,3407	0,4966
100	0,7702	0,7576	0,339	0,4694
200	0,7799	0,7603	0,3173	0,4667
500	0,8113	0,7625	0,3237	0,4646
1000	0,8071	0,7471	0,3468	0,4795
10000	0,8617	0,7201	0,3588	0,5043

- MAE-Wert zeigt ein Minimum bei 500; Da nur bis 500 Epochen trainiert wurde wird ein neues Training bis 1000 Epochen laufen gelassen



- Bis 1000 Epochen zeigt sich keine weitere Verbesserung -> Minimum hier bei circa 300 Epochen.
- In beiden Verläufen sieht man bis 300 Epochen eine Verringerung des MAE, daher sollten auf jeden Fall mehr als 100 Epochen trainiert werden (mindestens 300 weniger als 500 wegen overfitting)

Eintrag 8

Datum: 10.03.2026 **Titel:** Random Search **Aus Gruppentreffen:** Gruppentreffen 4 (27.02.2026)

Arbeitsschritte:

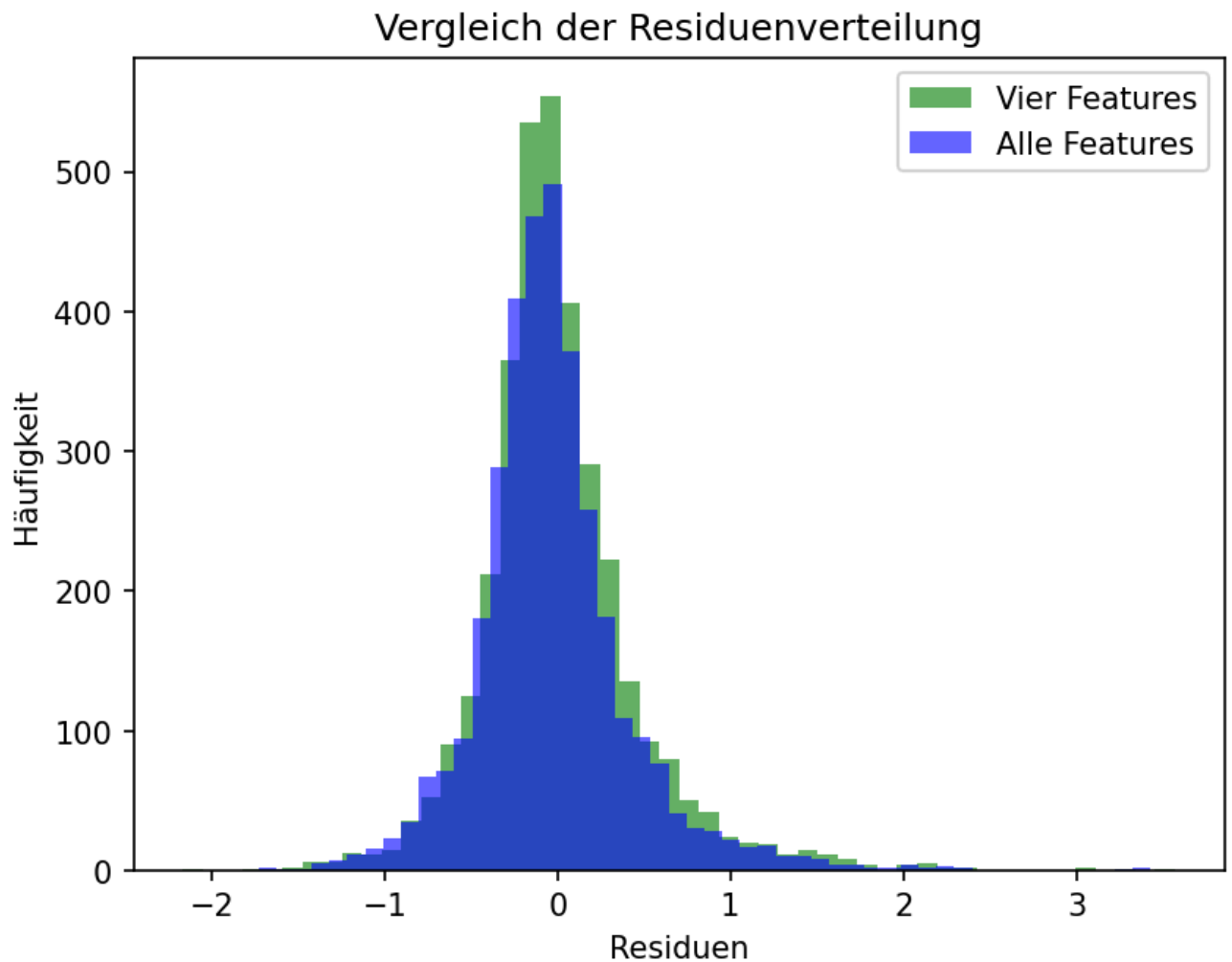
- Da sich die Hyperparameter gegenseitig beeinflussen muss ein finales Training auf einer Kombination von Hyperparametern basieren -> daher über verschiedene Hyperparameter testen mittels Random Search -> Wegen der benötigten Rechenzeit und zu erwarteten Ergebnissen wird RandomSearch GridSearch bevorzugt [5]
 - Bei über 100 Modellen konnte ein Test-Score von 0,7759 erreicht werden.
 - Da die Verbesserung an Hyperparametern zu stagnieren scheint werden alternativen ausprobiert. Nach Recherche wird LR-Schedule ausprobiert [10]
 - LR-Schedule zeigt keine signifikanten Verbesserungen
-

Eintrag 9

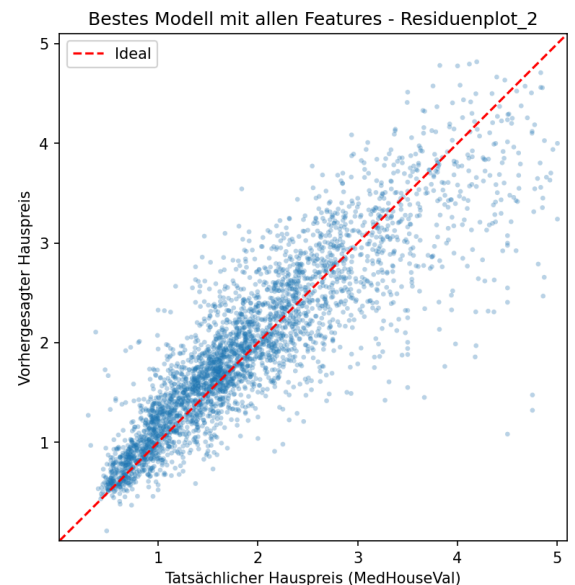
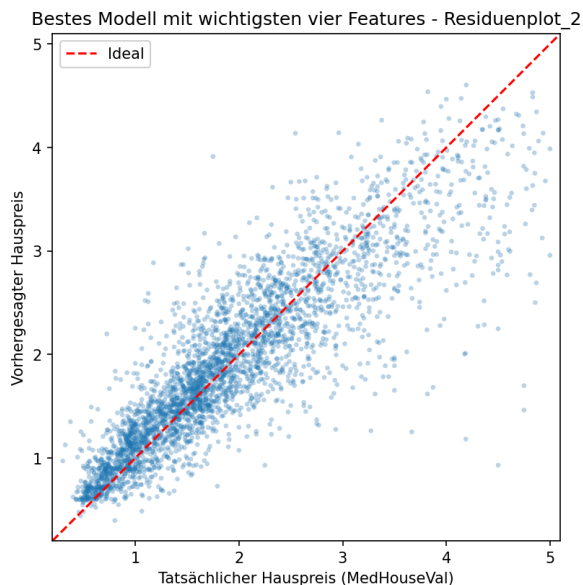
Datum: 11.03.2026 **Titel:** Feature Selektion und Qualitätsbeurteilung mittels Residuen Plots bzw. -Histogramme **Aus Gruppentreffen:** /

Arbeitsschritte:

- Variation von Hyperparametern zeigt keine wesentliche Verbesserung des Testscores, weshalb andere Wege gegangen werden (nicht in Gruppentreffen abgestimmt)
- Vergleich mit bereits trainierten Modellen, um Score einzuschätzen [9] -> Score liegt bei ungefähr 0,80 aber es gibt keine genauen Angaben zu Lernrate etc. -> Mehr Variation in der Netz Breite und Tiefe (wird allerdings parallel von Kolja untersucht laut Gruppentreffen 4, aber die Ergebnisse von Kolja sind noch nicht gepusht, daher kann ich Sie nicht übernehmen zum Training)
- Idee die Ergebnisse genauer zu Interpretieren um Rückschlüsse auf optimale Hyperparametern ziehen zu können, daher wird nur auf die wichtigsten Features trainiert um den Informationsgehalt zu bestimmen
- Training auf (MedInc, AveOccup, longitude, latitude) [8] zeigt bei 5 Modellen keine wesentliche Verschlechterung oder Verbesserung -> 5 Modelle sind sehr konsistent (weniger Freiheitsgrade beim Training; geringeres Overfitten) -> restlichen Features liefern keinen wesentlichen Informationsgewinn, bringen ebenfalls Rauschen mit ein und nur den selben Informationsgewinn -> vier Features tragen ein Großteil der erklärbaren Varianz -> Modellleistung scheint durch Datenqualität limitiert zu sein anstatt Modellkomplexität



Vergleich alle und vier wichtigsten Features_2



- Random Search nur auf vier Features angewendet, da es ein konsistenteres Modell verspricht
- Modellqualität beurteilen in Hinblick auf Hyperparameter: Residuen und Histogramme zeigen keinen Bias [6] -> eine Verbesserung wird hier erstmal nicht vorgeschlagen (Datenbereinigung ist in Ordnung) -> eventueller Ausreißer rechts unten genauer untersuchen, bei denen der tatsächliche Hauspreis deutlich niedriger ist, als der vorhergesagte Wert

- Cluster bei den Parametern Latitude und Longitude weisen eventuell auf ortsspezifische Korrelationen hin -> sollte später auch genauer untersucht werden
-

Eintrag 10

Datum: 12.03.2026 und 13.03.2026 **Titel:** Ensemble zur Stabilisierung von Ausreißern

Aus Gruppentreffen: Gruppentreffen 4 (27.02.2026)

Arbeitsschritte:

- Noch keine Information über Hyperparameter zur Netzstruktur, da entgegen Gruppentreffen 4 es zu einer Verzögerung kam. Kein Commit erkennbar.
- Parameterreduzierung, Dropout etc. soll von weiteren Gruppenmitgliedern übernommen werden.
- Suche nach weiteren Möglichkeiten, welche bereits noch nicht vergeben sind
- Mittels Ensemble sollen Ausreißer geglättet und das Modell stabilisiert werden [7]
- funktion `evaluate_predictions` nicht für Ensemble ausgelegt, da keine Trainingsdaten übergeben werden sollen, daher Funktion `evaluate_test_predictions()` geschrieben
- Fehler: `Outputs are collapsed...` (sollte eigentlich nur ein score ausgeben)
- Interpreter gesetzt aber numpy etc. wird in `evaluation.py` nicht erkannt
- `setting.json` angepasst, damit alle files den selben Kernel nutzen -> Fehler in `evaluation.py` weg, aber nicht die Lösung für das output collapse Problem
- Kernel ist abgestürzt und müsste neu gestartet werden -> beim ausführen der oberen Zellen ist aufgefallen, dass nicht die skalierten Daten verwendet wurden, sondern durch den einen Schleifendurchlauf die durch den `Min0_Max1` skalierten Daten, da der Score mittels `MinMax-Scaler` im Vergleich zum `StandardScaler` innerhalb der Varianz bei erneuter Ausführung liegt erwarte ich keine deutlichen Änderungen an den Hyperparametern
- Hyperparameter Lernrate, Batch-Size und Epochenanzahl sind betroffen -> Modelle neu trainieren zum Test
- Zeigt sich, dass die Standardisierung insgesamt den Score leicht verbessert, aber das Verhalten bezüglich Hyperparameter ist identisch wie mit den alten Daten
- output collapse konnte behoben werden, in der neuen Funktion in der Print ausgabe, war ein Fehler, weshalb 100 Zeichen hintereinander ausgegeben werden sollten und dadurch der output kollabiert ist
- Ensemble zeigt Verbesserung im Score (R^2 Test Score: 0,7983)

- Alle Kombinationen der Modelle vier bis dahin besten Modelle ausprobieren -> Je mehr Modelle in das Ensemble einfließen, desto besser ist der Score und wenn die Modelle unterschiedliche Featureanzahl haben wird der Score auch besser

```

Ensemble aus 2 Modellen:
Ensemble: Model 1 (8F-100iter) + Model 2 (4F)

=====
Ensemble: Model 1 (8F-100iter) + Model 2 (4F)
=====
R2 Score: Test = 0.7892
MAE:      Test = 0.3000
RMSE:     Test = 0.4377
=====
Ensemble: Model 1 (8F-100iter) + Model 3 (5F)

=====
Ensemble: Model 1 (8F-100iter) + Model 3 (5F)
=====
R2 Score: Test = 0.7879
MAE:      Test = 0.3013
RMSE:     Test = 0.4390
=====
Ensemble: Model 1 (8F-100iter) + Model 4 (8F-20iter)

=====
Ensemble: Model 1 (8F-100iter) + Model 4 (8F-20iter)
=====
R2 Score: Test = 0.7723
MAE:      Test = 0.3138
RMSE:     Test = 0.4549
=====
Ensemble: Model 2 (4F) + Model 3 (5F)

=====
Ensemble: Model 2 (4F) + Model 3 (5F)
=====
R2 Score: Test = 0.7797
MAE:      Test = 0.3057
RMSE:     Test = 0.4474
=====
Ensemble: Model 2 (4F) + Model 4 (8F-20iter)

=====
Ensemble: Model 2 (4F) + Model 4 (8F-20iter)
=====
R2 Score: Test = 0.7795
MAE:      Test = 0.3057
RMSE:     Test = 0.4477
=====
Ensemble: Model 3 (5F) + Model 4 (8F-20iter)

=====
Ensemble: Model 3 (5F) + Model 4 (8F-20iter)
=====
R2 Score: Test = 0.7754
MAE:      Test = 0.3086
RMSE:     Test = 0.4518
=====
Ensemble aus 3 Modellen:
Ensemble: Model 1 (8F-100iter) + Model 2 (4F) +
Model 3 (5F)

=====
Ensemble: Model 1 (8F-100iter) + Model 2 (4F) +
Model 3 (5F)
=====
R2 Score: Test = 0.7921
MAE:      Test = 0.2965
RMSE:     Test = 0.4346
=====
Ensemble: Model 1 (8F-100iter) + Model 2 (4F) +
Model 4 (8F-20iter)

=====
Ensemble: Model 1 (8F-100iter) + Model 2 (4F) +
Model 4 (8F-20iter)
=====
R2 Score: Test = 0.7901
MAE:      Test = 0.2989
RMSE:     Test = 0.4368
=====
Ensemble: Model 1 (8F-100iter) + Model 3 (5F) +
Model 4 (8F-20iter)

=====
Ensemble: Model 1 (8F-100iter) + Model 3 (5F) +
Model 4 (8F-20iter)
=====
R2 Score: Test = 0.7883
MAE:      Test = 0.3002
RMSE:     Test = 0.4386
=====
Ensemble: Model 2 (4F) + Model 3 (5F) + Model 4 (8F-20iter)

=====
Ensemble: Model 2 (4F) + Model 3 (5F) + Model 4 (8F-20iter)
=====
R2 Score: Test = 0.7893
MAE:      Test = 0.2978
RMSE:     Test = 0.4376
=====
Ensemble aus 4 Modellen:
Ensemble: Model 1 (8F-100iter) + Model 2 (4F) +
Model 3 (5F) + Model 4 (8F-20iter)

=====
Ensemble: Model 1 (8F-100iter) + Model 2 (4F) +
Model 3 (5F) + Model 4 (8F-20iter)
=====
R2 Score: Test = 0.7946
MAE:      Test = 0.2944
RMSE:     Test = 0.4321
=====

```

- Neue Modelle auf 2 und 3 Features trainiert um Ensemble zu erweitern -> Zu wenig Features, Modelle zu schlecht und auch keine Hilfe für Ensemble

- Mit Random Search werden fünf gute Modelle trainiert mit verschiedenen Hyperparametern und diese mit Ensemble zu einer Prediction zusammengeführt -> R^2 Test Score: 0,7861 (best_model_random_search_iterations_100.h5, best_model_4_important_features_iterations_5, best_model_5_important_features_iterations_5, best_model_6_important_features_iterations_5, best_model_random_search_iteration_20.h5)

```
=====
Ensemble Modell
=====
R² Score:  Test = 0.7983
MAE:      Test = 0.2891
RMSE:     Test = 0.4282
=====
```

- Ergebnisse daraus: Ensemble kann die Vorhersage erhöhen -> Modelle sind nicht redundant, Fehler sind nicht exakt korreliert
- Residuenplots zeigen große Ausreißer in Richtung eines zu niedrig vorhergesagten Hauspreises -> Ausblick: Diese Ausreißer im Datensatz suchen und auf eventuelle Fehler im Datensatz zurückführen. Eventuell Schwellwert bei Datenbereinigung herabsetzen

Eintrag 11

Datum: 14.03.2026 **Titel:** Git Probleme **Aus Gruppentreffen:** /

Arbeitsschritte:

- Commit nicht mehr möglich, durch nicht synchronisierte Änderungen
- Beim Zusammenlegen auf den Main Pfad waren alle Änderungen weg
- Änderungen wiederherstellen war nicht möglich, da eine Änderung bei 05_Neural_Network_Kolja vorgelegen hat (Merge Konflikt)
- Nach dem Merge konnten die alten Dateien wiederhergestellt werden

Eintrag 12

Datum: 19.03.2026, 20.03.2026 und 28.03.2026 **Titel:** E-Portfolio erstellen **Aus**

Gruppentreffen: Gruppentreffen 5 (17.03.2026)

Arbeitsschritte:

- Wie im Gruppentreffen 5 besprochen, wird eine Zusammenfassung am Ende des Notebook verfasst.
 - Einzelnde Codeabschnitte werden überarbeitet und die Resultate mit Markdown klar dergelegt
 - E-Portfolio wird angelegt
 - Zusammenfassung des Logbuchs wird verfasst
-

Eintrag 13

Datum: 31.03.2026 **Titel:** E-Portfolio beenden **Aus Gruppentreffen:** Gruppentreffen 6 (26.03.2026)

Arbeitsschritte:

- Nachdem Felix N. und Björn die Ergebnisse vorgestellt haben konnte die Einschätzung der Gruppenmitglieder und damit die Beendigung des e-Portfolios erfolgen
- nbstripout deinstallieren und wichtigsten Zellen erneut ausführen, damit der Code und die Abbildungen aus dem Git Repository nachvollziehbar sind (Aus Gruppentreffen 6)
- markdown (Logbuch) per Markdown pdf exportieren
- Kompilierungsfehler in Overleaf (timeout) lösen, indem ein Gratis 7 Tage Test gekauft wird